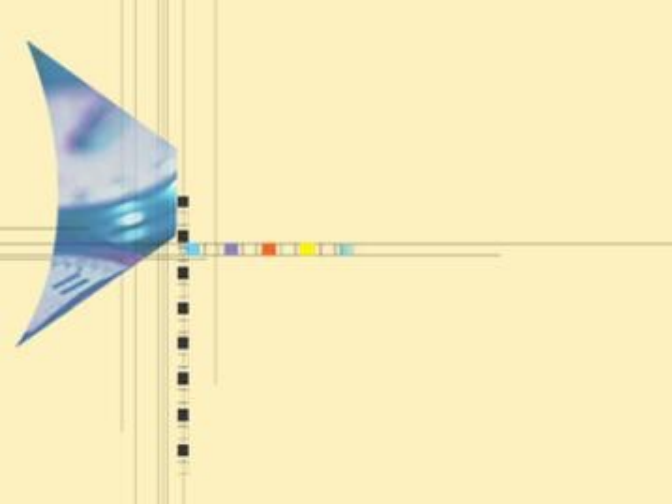




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

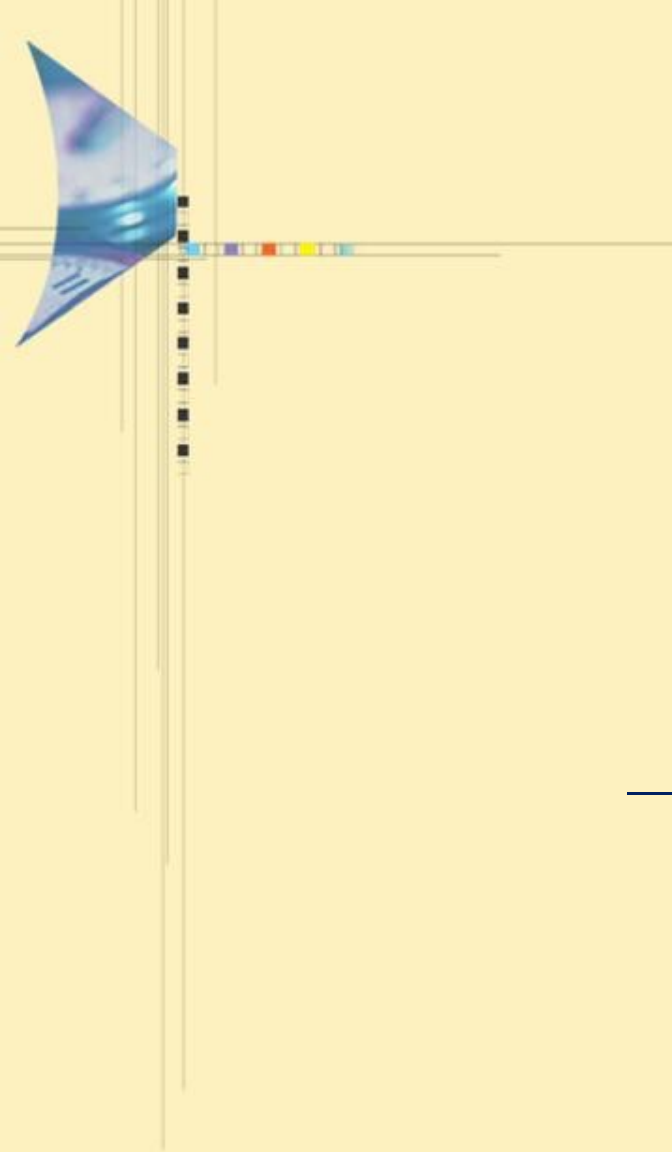
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/30

武汉大学计算机学院

武汉大学
Wuhan University





第十讲

随机算法 —Randomized Algorithms



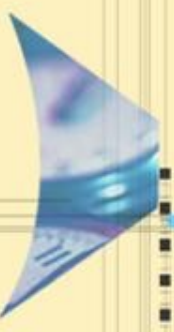
主要内容

10.1 引言

10.2 随机算法应用举例

10.3 随机游走算法





主要内容

10.1 引言

10.1.1 随机化算法

10.1.2 两类随机算法及其关系

10.1.3 随机算法的时间复杂度分析

10.1.4 随机算法的局限性



10.1 引言

- 随机算法把“对于所有合理的输入都必须给出正确的输出”这一求解问题的条件放宽，把随机性的选择注入到算法中，在算法执行某些步骤时，可以随机地选择下一步该如何进展，同时允许结果以较小的概率出现错误，并以此为代价，获得算法运行时间的大幅度减少。
- 随机算法优点
 - a) 与确定性算法相比，随机算法所需运行时间或空间通常较小；
 - b) 随机算法易于理解和实现。

举例1

- 假定有一个含 n 个变量的多项式表达式 $f(x_1, x_2, \dots, x_n)$ ，要核查 f 是否恒等于0。
- 用穷举分析的方法来检查可能是一项繁琐的工作。
- 采用随机的方式检查：
 - 生成一个随机 n 实数向量 $(r_1, r_2, \dots, r_n)$ ，并计算 $f(r_1, r_2, \dots, r_n)$ 的值。
 - 如果 $f(r_1, r_2, \dots, r_n) \neq 0$ ，那么 $f \neq 0$ ，假定失败；
 - 如果 $f(r_1, r_2, \dots, r_n) = 0$ ，那么 f 或是恒等于0，或是 $(r_1, r_2, \dots, r_n)$ 非常幸运。重复计算几次，继续得到 $f=0$ ，那么可以得出 f 恒等于0的结论，而且出错的可能性就很小。

举例2

- 设一个含有 n 个元素的数组，数组元素一半为字母‘a’，一半为字母‘b’。要求从数组中找出一个为‘a’的元素。
- 确定的查找策略：**直接从头到尾检查数组中元素**，直到找到字母‘a’为止。该方法可能需要最多 $n/2$ 次比较（例如，如果所有‘a’在‘b’之后）；或者**对数组中每2个元素检查**，同样最多可能需要 $n/2$ 次比较。。。
- 也就是说，无论采用哪种确定的查找策略，都无法保证算法对任何输入情况都能快速得到结果。
- 换一种思路，随机检查数组中的一个元素，那么无论实际的输入如何，都能以比较高的概率快速找到‘a’。

10.1.1 随机算法

- 随机算法：是在接收输入的同时，为了随机选择的目的，接收一串随机比特流且在运行过程中使用该比特流的算法。
- 一个随机算法在不同的运行中对于相同的输入可以有不同的结果（**随机的输出**）。由此得出，对于相同的输入，随机算法的两次运行时间可能有变化（**随机的性能**）。

10.1.2 两类随机算法及其关系

1) Las Vegas算法，即拉斯维加斯型算法

- 总是给出正确的解，或者无解。

2) Monte Carlo算法，即蒙特卡洛型算法

- 总是给出解，但偶尔可能是错误解
- 可以通过多次运行原算法，且满足每次运行时随机选择都相互独立，使产生错误解的概率减到任意小。

注意：Las Vegas和Monte Carlo不是一种算法的名称，而是对一类随机算法的特性的概括。

两个简单的算法例子

Monte Carlo 算法

- 令 $A[1..n]$ 是一个由 n 个不同数构成的数组， n 是偶数。希望选择一个数 x 满足大于中间元。

算法 10.1 MC

1. 令 $x \leftarrow -\infty$
2. 重复第3-第4步 k 次:
3. 从 A 中取样一个元素 y
4. 如果 $y > x$, 则 $x \leftarrow y$
5. 返回 x

概率与正确性
有关，运行时间
是确定的

两个简单的算法例子

Las Vegas算法

- 令 $A[1..n]$ 是一个由 n 个不同数构成的数组， n 是偶数。 A 含有 $(n/2)+1$ 个相同元素 x ，且 $(n/2)-1$ 个互异的元素都不同于 x ，希望找到这个重复的 x 。

算法10.2 LV

1. 重复执行不确定次数的第2和第3步。
2. 从下标集 $\{1, 2, \dots, n\}$ 中取样 i 和 j
3. 如果 $i \neq j$ 且 $A[i] = A[j]$ ，则 $x \leftarrow A[i]$ 且退出。

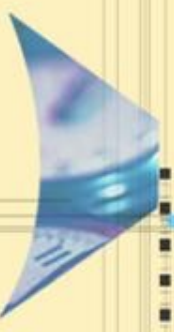
概率与运行时间有关，结果总是正确的

拉斯维加斯型随机算法

- 对于拉斯维加斯算法，通常需要分析算法的平均运行时间。运行时间本身是一个随机变量。
- 期望运行时间是输入规模的多项式且总能给出正确答案的随机算法称为有效的拉斯维加斯算法。
- 有效的拉斯维加斯算法也称为ZPP类（zero-error probabilistic polynomial time）算法，即零错误概率多项式时间随机算法。

蒙特卡洛型随机算法

- 对于蒙特卡洛型随机算法，通常需要分析算法的**出错概率**。蒙特卡洛型随机算法的运行时间和出错概率都是随机变量。
- 总是在多项式时间内运行且出错概率不超过 $1/3$ （可以任意小）的随机算法称为有效的蒙特卡洛型随机算法。
- 有效的蒙特卡洛型随机算法也称为BPP类算法（bounded-error probabilistic polynomial time），即错误概率有限的多项式时间随机算法。



蒙特卡洛型随机算法的出错类型

- 弃真型错误：就是在求解一个判定问题时把本应该接受的输入误判为拒绝。
- 取伪型错误：就是把本应该拒绝的输入误判为接受。

蒙特卡洛型随机算法的出错类型

- 根据是否同时出现两类错误，蒙特卡洛型算法又分为单侧错误和双侧错误两类：
 - 单侧错误的蒙特卡洛型算法：在所有输入上，要么犯弃真的错误，要么犯取伪的错误，不同时出现两种不同的错误。
 - 双侧错误的蒙特卡洛型算法：在所有输入上可以同时出现两种不同的错误。

单侧错误蒙特卡洛型随机算法

- 对于弃真型的单侧错误随机算法来说，当算法宣布接受时，结果一定是对的；而当算法宣布拒绝时，结果有可能是错误的；有效的弃真型单侧错误随机算法称为RP类算法；
- 对于取伪型的单侧错误随机算法来说，当算法宣布拒绝时，结果一定是对的，而当算法宣布接受时，结果有可能是错误的。有效的取伪型单侧错误随机算法称为coRP类算法。

Las Vegas与Monte Carlo对比

	拉斯维加斯算法	蒙特卡罗算法
例子	从一串钥匙中试出能开锁的钥匙	从不透明的苹果筐中挑最大的苹果
策略	尽量找，但不保证能找到（除非全枚举）	尽量找一个最大的，但不保证是最大的
规律	采样越多，越有可能找到正确解	采样越多，越可能得到正确解（最大的）
适用情景 (依问题而定)	对采样要求没有限制，要求必须给出正确解	要求在有限采样内，必须给出一个解，但不要求是正确解

10.1.3 随机算法的时间复杂度分析

- 对于一个确定算法A，衡量算法A的时间复杂度的一个指标是它的平均运行时间。当每个大小为n的输入被认为具有同等可能性时，A所花费的平均时间。也就是说，假设所有输入都是均匀分布的，平均时间就是所有输入情况运行时间的平均。但是，大小为n的不同输入可能不是均匀分布的。因此平均运行时间分析简单理解为求平均可能误导，应理解为是期望运行时间。
- 对于随机算法A，其在大小为n的一个实例I上的运行时间可能因不同次执行而异。因此，更自然的性能度量是A在实例I上的期望运行时间，也即算法A反复求解实例I所花费的平均时间。

随机算法的期望运行时间 vs 确定算法的平均情况时间

- 确定算法的平均情况分析中，假设所有输入上有一个概率分布，算法在不同输入上的运行时间是随机变量，这个随机变量的期望就是算法的平均运行时间。
 - 平均情况分析中，给定一个算法，概率分布定义在输入上。
- 在随机算法中，在给定一个输入上，随机算法的运行时间是随机变量，这个随机变量的期望值是期望运行时间；
 - 随机算法中给定一个输入和一个随机序列后，算法的执行时间就是确定的。因此一个随机算法等价于一簇确定算法以及这簇确定算法的一个概率分布。因此随机算法给定一个输入，概率分布定义在算法上。

期望时间

- 期望时间：对于大小为n的特定输入I，算法的期望时间就是在随机数的不同序列上的时间的平均：

$$ET(A, I) = \sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R)$$

- 最坏情况下的期望时间：算法的最坏情况下的期望时间是所有大小为n的输入上的期望时间的最大值。

$$W_ET(A) = \underset{\text{input } I \text{ with size } n}{MAX} \left(\sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R) \right)$$



10.1.4 随机算法的局限性

- 根据计算复杂性理论有关结论，不太可能为NP完全问题设计出多项式时间的随机算法。
- 尽管如此，对于很多问题，随机算法依然可能比最好的确定算法更快。